

Task 2: Multi-class Classification using FCNN on Fashion-MNIST

Optimizer Comparison Report

1. Objective

Train fully connected neural networks (FCNNs) with 3–5 hidden layers using six optimizers and compare convergence and classification performance.

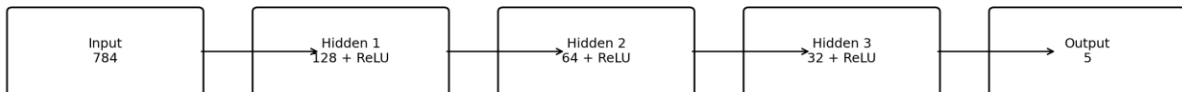
2. Dataset and Preprocessing

Item	Value
Dataset	Fashion-MNIST
Selected classes	Trouser, Sandal, Sneaker, Bag, Ankle boot
Selected samples	35,000
Image size	28×28 grayscale
Input size	784
Normalization	Pixel values / 255 $\rightarrow$ $[0, 1]$
Final split	64% train (22,400), 16% validation (5,600), 20% test (7,000)
Split	Stratified; random seed = 42
Output classes	5
Device	CUDA

The held-out test set was not used for model selection.

3. FCNN Architectures

Arch-A (3 layers): $784 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 5$ | Trainable parameters = {total:,}



Arch-B (4 layers): $784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 5$ | Trainable parameters = {total:,}



Arch-C (5 layers): $784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 5$ | Trainable parameters = {total:,}

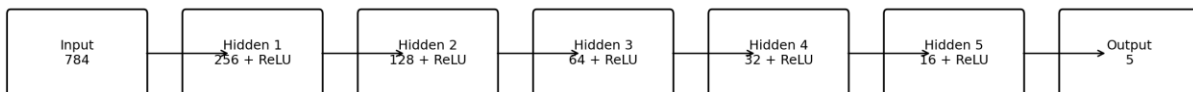


Figure 1. FCNN architectures

Architecture	Layer structure	Trainable parameters
Arch-A	784 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 5	110,981
Arch-B	784 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 5	244,357
Arch-C	784 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 5	244,805

Trainable parameters are computed as $\Sigma(n_{in} \times n_{out} + n_{out})$ for all fully connected layers.

4. Loss, Optimizers and Hyperparameters

Cross-entropy loss:

$$L = -(1/N) \sum_i \log p_i, y_i$$

Softmax output: $p_k = \exp(z_k) / \sum_j \exp(z_j)$

Fully connected layer: $z^l = W^l a^{l-1} + b^l$; hidden activation: $a^l = \text{ReLU}(z^l)$

Parameter	Value used in completed notebook
Learning rate η	0.001
Momentum (Momentum / NAG)	0.9
RMSProp β (alpha)	0.99
RMSProp ϵ	1×10^{-8}
Adam β_1	0.9
Adam β_2	0.999
Adam ϵ	1×10^{-8}
Stopping threshold	1×10^{-4}
Maximum epochs	100 (non-Batch-GD); 5000 (Batch GD)
Mini-batch size	16 for SGD, Momentum, NAG, RMSProp, Adam
Batch GD size	22,400 (all training samples)

Optimizer update forms:

- SGD: $\theta_{t+1} = \theta_t - \eta g_t$
- Momentum: $v_t = 0.9v_{t-1} + g_t$; $\theta_{t+1} = \theta_t - \eta v_t$
- NAG: SGD with momentum = 0.9 and nesterov=True.
- RMSProp: $v_t = 0.99v_{t-1} + 0.01g_t^2$; $\theta_{t+1} = \theta_t - \eta g_t / (\sqrt{v_t} + \epsilon)$
- Adam: $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$; $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$, followed by the Adam parameter update.

Stopping criterion: $|E_t - E_{t-1}| < 1 \times 10^{-4}$, where E is average training cross-entropy error.

5. Convergence and Accuracy Results

Architecture	Optimizer	Epochs	Train Acc. (%)	Validation Acc. (%)
Arch-A (3 layers)	SGD	69	97.05	95.89
Arch-A (3 layers)	Batch GD	2	20.00	20.00
Arch-A (3 layers)	Momentum	30	98.99	96.89
Arch-A (3 layers)	NAG	30	98.80	96.77
Arch-A (3 layers)	RMSProp	49	99.71	97.18
Arch-A (3 layers)	Adam	25	99.46	97.36
Arch-B (4 layers)	SGD	84	97.75	95.96
Arch-B (4 layers)	Batch GD	2	20.00	20.00
Arch-B (4 layers)	Momentum	45	99.44	97.11

Arch-B (4 layers)	NAG	37	98.42	96.43
Arch-B (4 layers)	RMSProp	33	99.47	97.21
Arch-B (4 layers)	Adam	22	98.87	96.98
Arch-C (5 layers)	SGD	73	97.03	94.77
Arch-C (5 layers)	Batch GD	2	20.00	20.00
Arch-C (5 layers)	Momentum	27	99.09	96.82
Arch-C (5 layers)	NAG	44	99.49	97.12
Arch-C (5 layers)	RMSProp	64	99.54	96.82
Arch-C (5 layers)	Adam	54	99.69	97.25

6. Training Error Curves

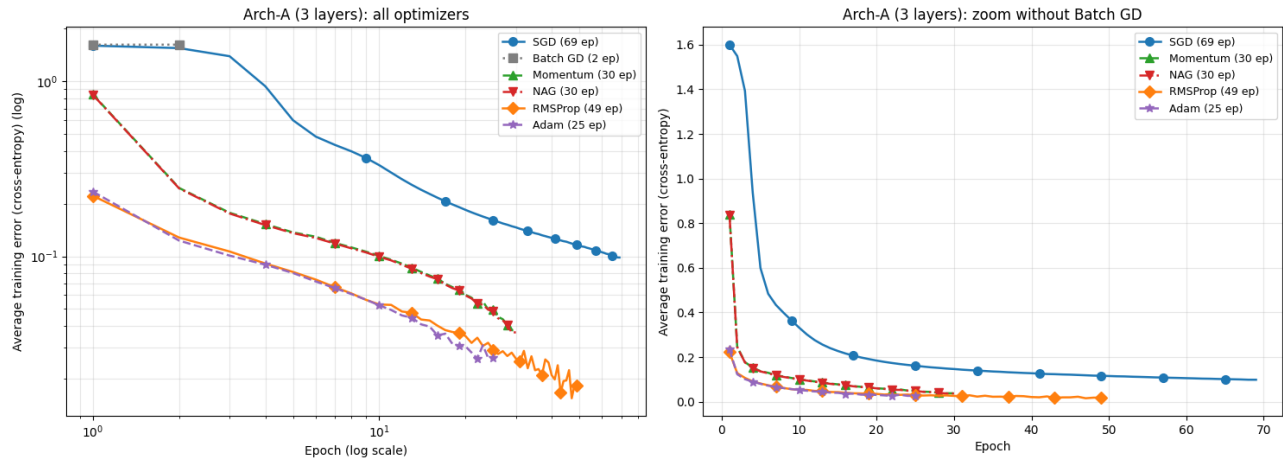


Figure 2. Arch-A training error curves for all optimizers and the zoomed view without Batch GD.

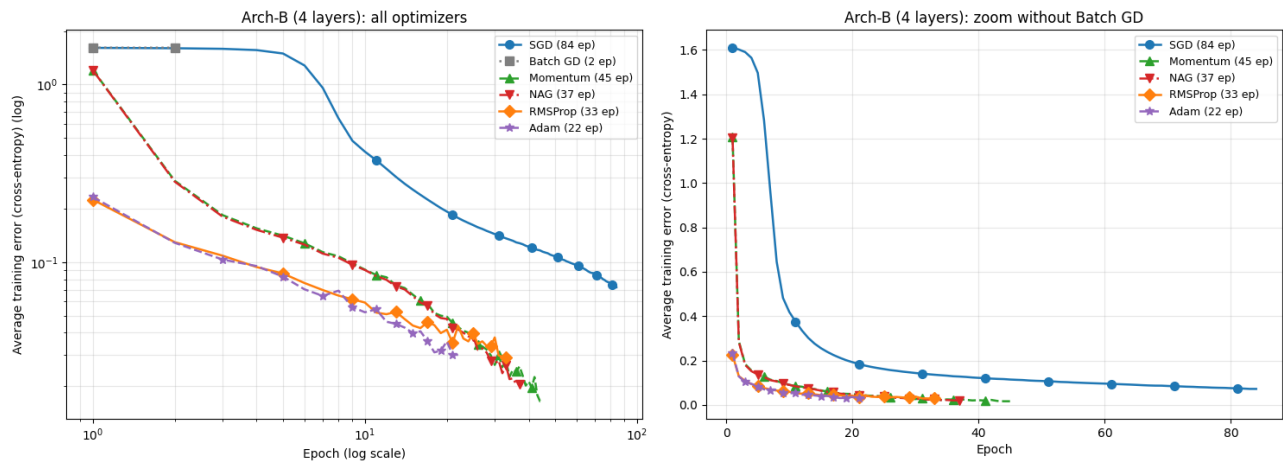


Figure 3. Arch-B training error curves for all optimizers and the zoomed view without Batch GD.

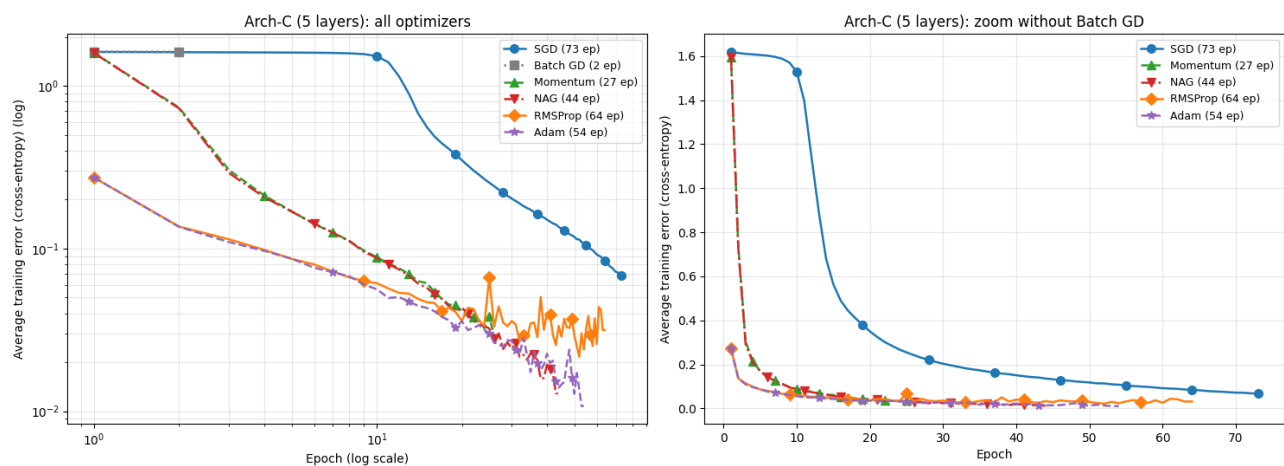


Figure 4. Arch-C training error curves for all optimizers and the zoomed view without Batch GD.

7. Best Architecture

Item	Result
Best architecture	Arch-A (3 layers): 784 → 128 → 64 → 32 → 5
Best optimizer	Adam
Validation accuracy	97.36%
Training accuracy	99.46%
Test accuracy	97.64%

Selection criterion: highest validation accuracy across all 18 architecture–optimizer combinations.

8. Final Confusion Matrices

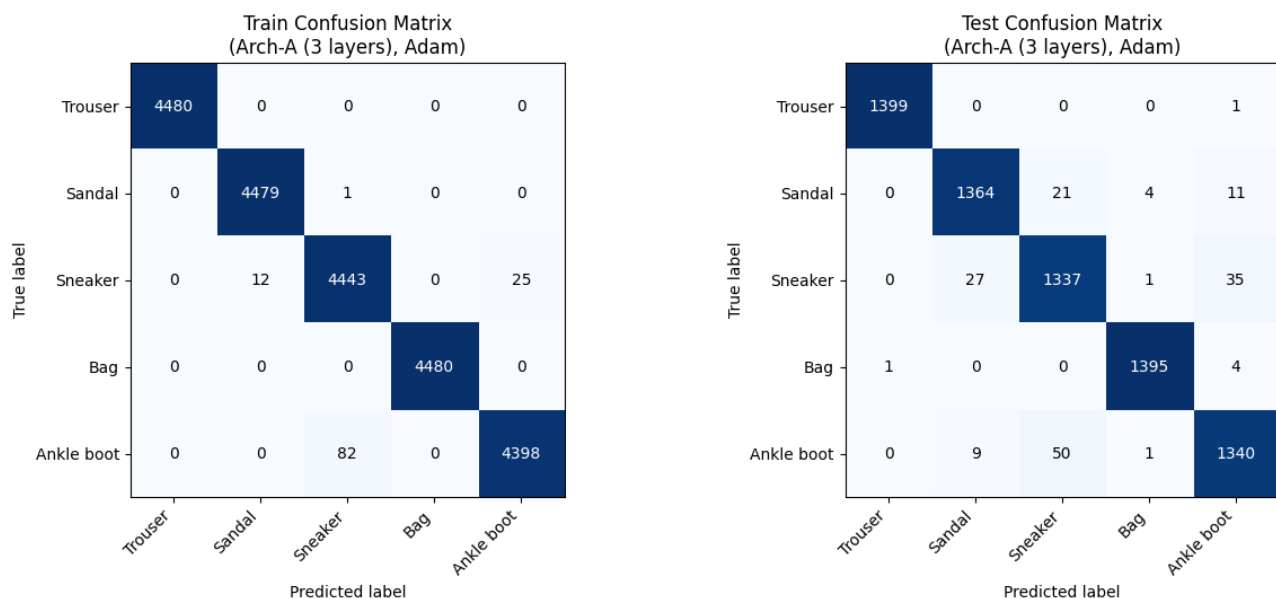


Figure 5. Training and test confusion matrices for Arch-A + Adam.

Training confusion matrix (rows = true class, columns = predicted class):

	Trouser	Sandal	Sneaker	Bag	Ankle boot
Trouser	4480	0	0	0	0

Sandal	0	4479	1	0	0
Sneaker	0	12	4443	0	25
Bag	0	0	0	4480	0
Ankle boot	0	0	82	0	4398

Test confusion matrix (rows = true class, columns = predicted class):

	Trouser	Sandal	Sneaker	Bag	Ankle boot
Trouser	1399	0	0	0	1
Sandal	0	1364	21	4	11
Sneaker	0	27	1337	1	35
Bag	1	0	0	1395	4
Ankle boot	0	9	50	1	1340

9. Final Results

- Best validation performance: Arch-A + Adam = 97.36%.
- Final training accuracy of the selected model: 99.46%.
- Final test accuracy of the selected model: 97.64%.
- Batch GD reached the stopping criterion in 2 epochs for all three architectures, but its recorded accuracy remained at 20.00%.
- The selected model used the smallest architecture: 110,981 trainable parameters.